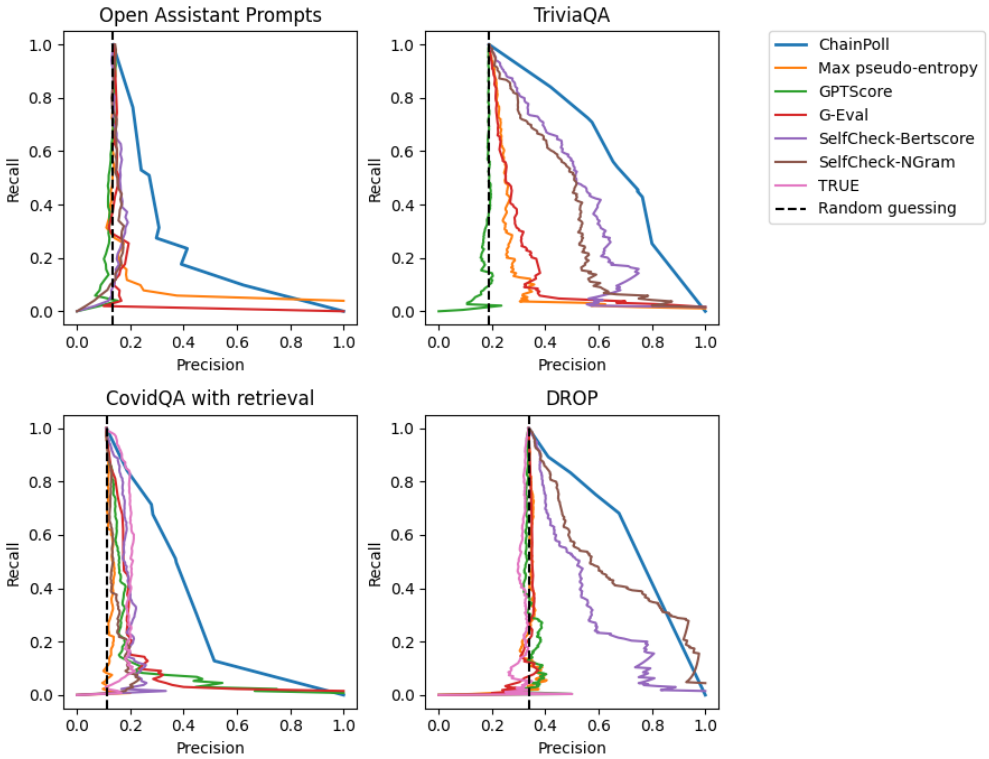

Figure 2: Precision-recall curves for hallucination detection across *RealHall* datasets. The ChainPoll curves are *ChainPoll-Correctness* in the top row, and *Chainpoll-Adherence* in the bottom row.

6 Related work

The field of LLM hallucination detection is relatively new, as LLMs themselves are relatively new.

Rather than giving a complete historical review of this field, we will cover the specific metrics that we deemed most promising when reviewing the literature.

Table 5 provides a summary view of these metrics, comparing and contrasting them to our best-performing metric, *ChainPoll*.

Metric	Description	Tested against∗	Type[a]	Cost/ex[b]	Batch[c]	GPU-free[d]	Quality vs ours[e]
ChainPoll (ours)	Prompting + aggregation	*RealHall*	OC	5	✓	✓	100%
SelfCheck-BertScore [1]	Sentence-level rerun checking	Wikipedia articles	O	>20	✓		63%
SelfCheck-NGram [1]	Sentence-level rerun checking	Wikipedia articles	O	20	✓	✓	52%
TRUE NLI [4]	NLI	Various (closed-domain)	C	≤ 1	✓		33%[h]
G-Eval-3.5[f] [3]	Prompting + aggregation	SummEval, TopicalChat	C	20	✓	✓	29%
GPTScore [2]	Prompting + perplexity	Various (closed-domain)	C	≤ 1	✓	✓	9%
SelfCheck-MQAG [1]	Sentence-level rerun checking	Wikipedia articles	O	>20			N/A[g]
ChatProtect [17]	Sentence-level rerun checking	Wikipedia articles	O	2/sentence		✓	N/A[g]

Table 5: How our metrics compare to others in the literature. ∗ This column lists the evaluation data used to test the quality of the metric in the original publication introducing it. In this paper, we independently evaluate all metrics on *RealHall*. [a] O = open-domain, C = closed-domain, OC = both open-domain and closed-domain [b] An estimate of how compute-intensive the metric is, in units of *additional LLM-generated responses* required during evaluation of a single response. $> N$ denotes a metric which generates N completions and then does additional computation with a neural model. [c] Whether the computations noted in the *Cost/ex* column can be computed in parallel. In practice, batch metrics are much faster than sequential metrics, even if they require more computation per example. [d] Whether the metric can be served without the added expense of a dedicated GPU. [e] Average (AUROC score minus 0.5) over datasets in *RealHall*, as a fraction of *ChainPoll*'s performance. We subtract 0.5 to normalize scores because an AUROC score of 0.5 corresponds to random guessing. [f] We do not benchmark G-Eval-4, as it does not meet our bar for cost (it is 20x as expensive as the already-expensive GPT-4). [g] We did not benchmark these metrics, as their high compute intensity and sequential nature do not meet our bar for efficiency. [h] Closed-domain tasks only, as TRUE NLI cannot be applied in open-domain tasks.

6.1 SelfCheckGPT

SelfCheckGPT [1] proposed an approach based on checking the self-consistency between an LLM response and a large number of additional responses, sampled from the same LLM using the same prompt. For their main experiments, the authors used 20 additional responses per evaluated response.

The authors introduced three metrics using this approach, which differ in the way they compute agreement between responses.

- *SelfCheck-BertScore* computes agreement using BertScore [18].
- *SelfCheck-NGram* computes agreement by fitting a simple unigram[7] language model and using its probabilities on the original response.
- *SelfCheck-MQAG* computes agreement using a MQAG [19], a complex question-answering pipeline using four fine-tuned neural models. The pipeline generates multiple-choice questions based on the original response, then tries to answer them using only the additional responses.

Notably, the SelfCheckGPT metrics were proposed as *sentence-level* metrics, requiring computation of agreement scores separately for each sentence in the response – which can be computationally expensive for long responses. (This expense combines with the expense of generating a potentially large number of additional responses.)

To compute response-level aggregates, the scores for sentences are averaged.

The SelfCheckGPT metrics were evaluated on a dataset of 238 prompts written by the authors, all of which ask the model to write a Wikipedia page for a specific person. We critically assess this dataset in Section A.1.1.

[7]The authors experimented with different gram lengths, finding that unigrams worked best.

6.2 G-Eval

G-Eval [3] propose an approach that evaluates an LLM response by asking an LLM[8] to rate the response on a 1-5 scale, with provided guidelines.

The LLM's probabilities of outputting the tokens `' 1'`, `' 2'`, `etc.` are used to produce a weighted-average rating. When probabilities are not available, the authors sample from the LLM 20 times and average the ratings over this sample.

G-Eval was evaluated on two closed-domain datasets, SummEval and TopicalChat. We critically assess these datasets in Section A.1.1, with further analysis in Appendix A.6.

6.3 GPTScore

GPTScore [2] uses a simple method which evaluates an LLM response by prepending a instruction (e.g. "write a factually consistent summary") to the prompt and response, then evaluating the *perplexity* of the response using an LLM[9].

GPTScore was evaluated on a large number of closed-domain datasets, such as SummEval.

We reproduce GPTScore's strong performance on SummEval (Appendix A.6), yet we find that it performs very poorly on *RealHall*.

We hypothesize that this discrepancy results from the following:

- We find that the prefix adds little value: we can achieve nearly identical performance on SummEval by simply computing perplexity on the original response.
- We analyze the strong performance of perplexity on SummEval, finding that it results from a mismatch between the (weaker) models used to generate responses in SummEval and the (stronger) models used to compute perplexity.
- When a strong, modern LLM is used *both* to generate responses and to compute perplexity, the strong performance of perplexity (and thus GPTScore) disappears.

6.4 TRUE

TRUE [4] builds a benchmark suite of 11 closed-domain datasets, covering similar ground to the evaluation datasets used in GPTScore [4].

The authors compare a number of different metrics on these datasets. Their best-performing metric used probabilities from a T5-XXL model finetuned for natural language inference (NLI).

In conjunction with the paper, the authors released the weights of a model similar to this one, though trained on a different data mixture. We compute scores using the released model, and refer to this metric as *TRUE NLI*.

6.5 ChatProtect

ChatProtect [17] proposes an approach similar to SelfCheckGPT [1].

Like SelfCheckGPT, ChatProtect works on the sentence level, and uses self-consistency between multiple responses to detect hallucinations.

Whereas SelfCheckGPT generates alternatives at the *response*, ChatProtect generates a separate alternative version of each *sentence* in context, and checks each one for consistency against the original sentence.

7 Conclusion

Hallucinations are possibly the single largest impediment to widespread practical use of LLMs. This fact means there is a pressing need to identify ways of automatically discovering hallucinations in LLM outputs.

[8] Either a different LLM, or the same one.

[9] As in G-Eval, this may be a different LLM, or the same one.